

绘图库Matplotlib



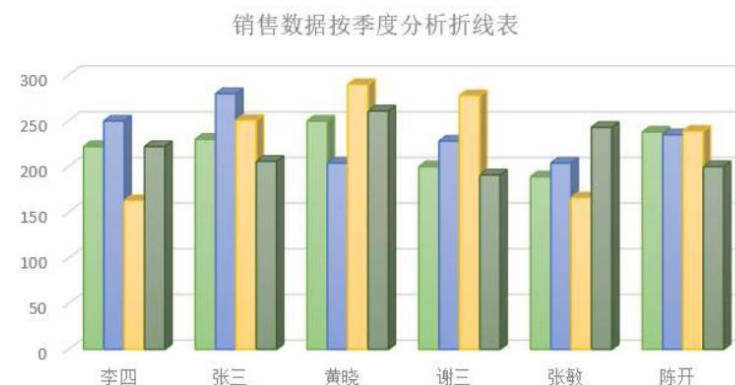
本章内容

- 数据可视化是什么?
- Matplotlib是什么?
- Matplotlib下载和安装?
- 第一个Matplotlib绘图程序?
- matplotlib的基本方法

+ 数据可视化是什么

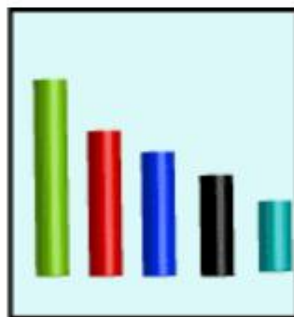
- 顾名思义，数据可视化就是将数据转换成图或表等，以一种更直观的方式展现和呈现数据。通过“可视化”的方式，我们看不懂的数据通过图形化的手段进行有效地表达，准确高效、简洁全面地传递某种信息，甚至帮助我们发现某种规律和特征，挖掘数据背后的价值
- 举个例子:

1线	BCA001-47640	20	50	50	5.83	3.00	9.67	5596	15	14	82940	0	12585	89.80%	12148	
2线	BCA001-45163			0	0.00	11.00	11.00	1001	119	1	119119	0	10829	74.60%	10829	
1线	BCA001-41954	30	60	75	6.75	3.67	8.88	1692	29	16	35	49068	59220	16356	116.83%	16407
2线	BCA001-48680	21	50	52.5	6.13	2.50		1840	31	10	37	57040	68080	15542	111.02%	14407
1线	BCA001-34467	45		0	0.00	11.00	11.00	1580	25	10	29.5	39500	46610	15800	112.86%	13625
2线	BCA001-34467			0	0.00	11.00	11.00	1204	119	1	143276	0	13025	89.83%	13025	
1线	BCA001-16137	30	35	75	6.75	2.75	6.08	1010	21	10	24.5	21210	24745	11105	79.32%	15036
2线	BCA001-16137	22	120	60	55	20.42	2.33	1300	24	5	27.5	31200	35750	13391	95.65%	16761
1线	BCP147-2833154	7	40	17.5	2.04	4.00	9.58	2000	19	6	38000	0	9500	67.86%	9443	
2线	BCP147-2833154	19	30	17.5	5.54	2.33		2500	7	20	17500	0	7511	53.65%	9443	
1线	BCP147-2833154	8	30	20	2.33	5.00		2500	14	20	35000	0	11667	83.33%		

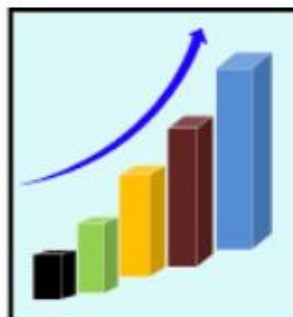


+ 数据可视化

- 图表为更好地探索、分析数据提供了一种直观的方法，它对最终分析结果的展示具有重要的作用。



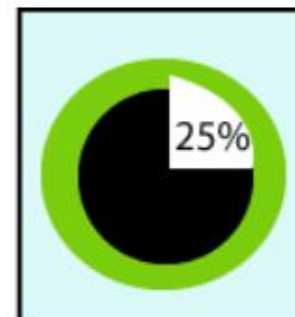
柱状图



直方图



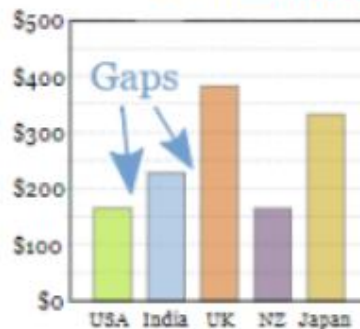
散点图



饼状图

- 柱状图=条形图

柱状图/条形图



← 分类数据 →

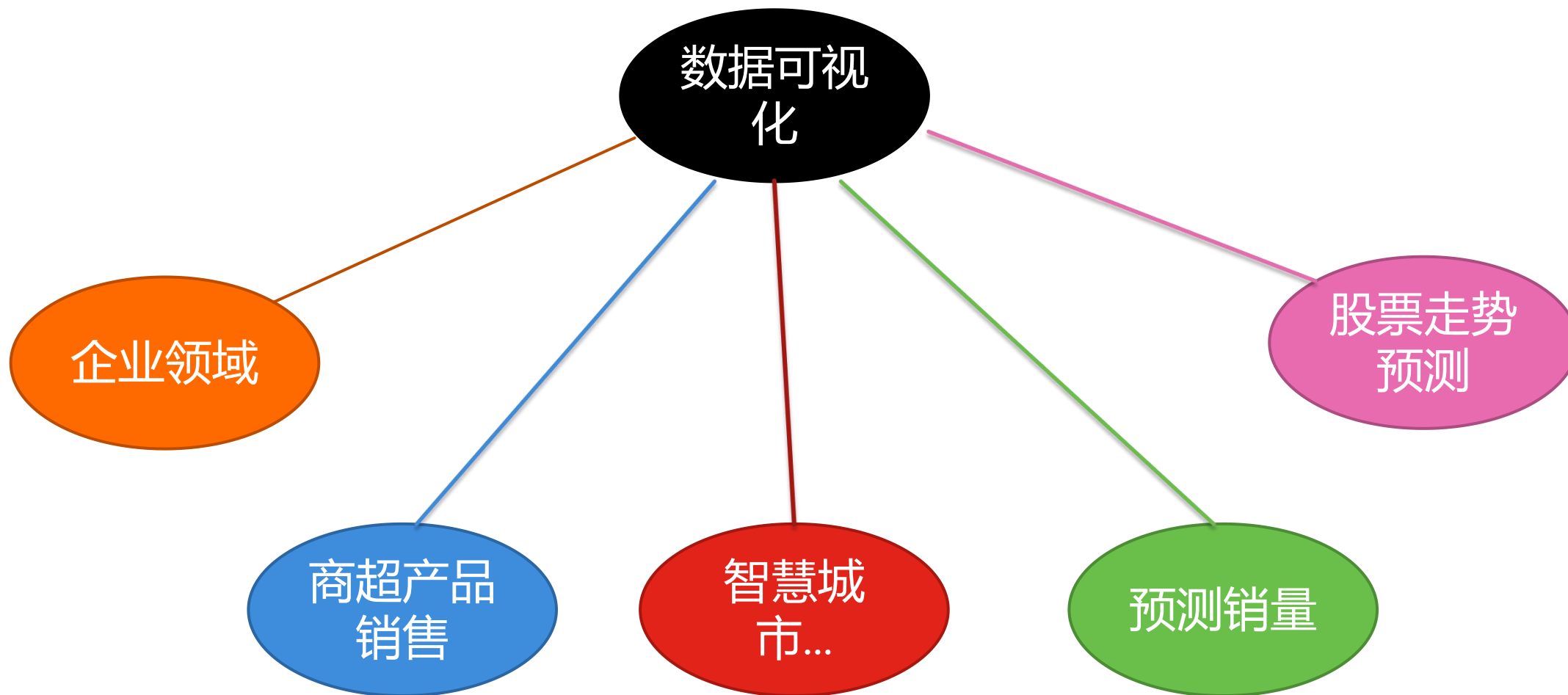
直方图



← 数字区间 →

- 直方图=特殊的条形图

➤ 数据可视化应用场景



➤ 可视化工具选择



- 是一款用于数据可视化的 Python 软件包，支持跨平台运行，它能够根据 NumPy ndarray 数组来绘制 2D(3D) 图像，它使用**简单、代码清晰易懂**，深受广大技术爱好者喜爱。
- 优点:
 - Matplotlib 提供了一个套面向绘图对象编程的 API 接口
 - 依托于Python，借助Python的强大的可拓展性，Matplotlib可以在许多不同的环境中被使用
 - 基于Matlab的和基于面向对象的,但是它完全免费(Matlab是商业数学软件)
 - Matplotlib 实现了几乎是完全自主控制的图形定义功能

➤ Matplotlib下载和安装

- 使用pip管理器安装

```
pip install matplotlib
```

- 使用Anaconda安装



- 验证安装

```
>import matplotlib  
>matplotlib.__version__  
>'3.1.3'
```

+ 第一个Matplotlib绘图程序

- 首先导入 Matplotlib 包中的 Pyplot 模块

```
import matplotlib.pyplot as plt
```

- 接下来，使用 NumPy 提供的函数 `arange()` 创建一组数据来绘制图像。

```
#引入numpy包  
import numpy as np  
#获得0到50之间的ndarray对象  
x = np.arange(0, 50)
```

- 上述所得 `x` 的值作用到 `x` 轴上，而该值的平方值，也就是 `y` 值,使用以下方式获取

```
y = x **2
```

- 使用 `plot()` 函数对 `x`、`y` 进行绘制。

```
plt.plot(x,y)
```


+ matplotlib的基本方法

方法名	说明
<code>title()</code>	设置图表的名称
<code>xlabel()</code>	设置x轴名称
<code>ylabel()</code>	设置y轴名称
<code>xticks(ticks,label,rotation)</code>	设置x轴的刻度,rotation旋转角度
<code>yticks()</code>	设置y轴的刻度
<code>show()</code>	显示图表
<code>legend()</code>	显示图例
<code>text(x,y,text)</code>	显示每条数据的值 x,y值的位置



matplotlib基本方法

title()
设置图表的名称

xlabel()/ylabel()
设置x轴/y轴名称

xticks
设置x轴的刻度

中文设置

plt.rcParams['font.sans-serif']=['SimHei']

plt.rcParams['axes.unicode_minus']=False

show()
显示图表

legend()
设置图例

loc位置

text(x,y,text)

x:x坐标

y:y坐标

text:显示的信息

➕ 其他元素可视性

- 显示网格:plt.grid()
 - `plt.grid(True, linestyle = "--",color = "gray", linewidth = "0.5",axis = 'x')`
 - 显示网格
 - `linestyle`: 线型
 - `color`: 颜色
 - `linewidth`: 宽度
 - `axis`: x, y, both, 显示x/y/两者的格网

➕ 其他元素可视性

- 显示网格:plt.grid()
 - `plt.grid(True, linestyle = "--",color = "gray", linewidth = "0.5",axis = 'x')`
 - 显示网格
 - `linestyle`: 线型
 - `color`: 颜色
 - `linewidth`: 宽度
 - `axis`: x, y, both, 显示x/y/两者的格网

⊕ 其他元素可视性

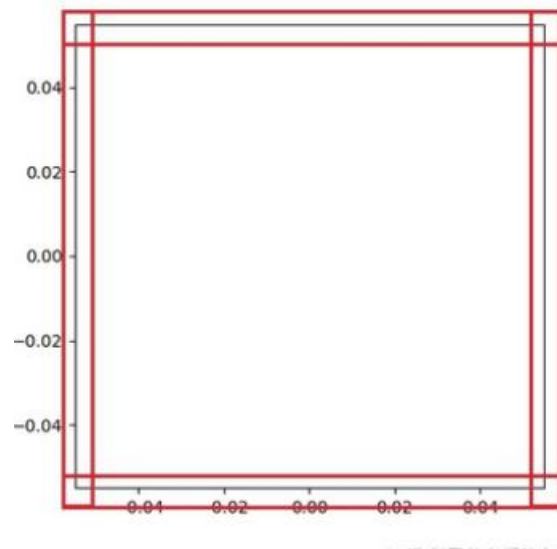
- 显示网格:plt.grid()
 - `plt.grid(True, linestyle = "--",color = "gray", linewidth = "0.5",axis = 'x')`
 - 显示网格
 - `linestyle`: 线型
 - `color`: 颜色
 - `linewidth`: 宽度
 - `axis`: x, y, both, 显示x/y/两者的格网

➕ 其他元素可视性

- 显示网格:plt.grid()
 - `plt.grid(True, linestyle = "--",color = "gray", linewidth = "0.5",axis = 'x')`
 - 显示网格
 - `linestyle`: 线型
 - `color`: 颜色
 - `linewidth`: 宽度
 - `axis`: x, y, both, 显示x/y/两者的格网

➤ 对坐标轴的操作

- `plt.gca()` # 获取当前坐标轴
 - 首先观察画布上面的坐标轴，如下图



- 上图中，用红色标识出的黑色边界框线在Matplotlib中被称为**spines**，中文翻译为脊柱.....在我理解看来，意思是这些边界框线是坐标轴区域的“支柱”。
- 那么，我们最终要挪动的其实就是这四个“支柱”
- 且所有的操作均在`plt.gca()`中完成，`gca`就是get current axes的意思

➤ 设置画图的分辨率，大小等信息

- `plt.rcParams['figure.figsize'] = (8.0, 4.0)` # 设置figure_size尺寸
- `plt.rcParams['figure.dpi'] = 300` #分辨率
 - 默认的像素: [6.0,4.0], 分辨率为72, 图片尺寸为 432x288
 - 指定dpi=100, 图片尺寸为 600*400
 - 指定dpi=300, 图片尺寸为 1800*1200

➤ 图表的样式参数设置

● 1.线条样式

```
plt.plot(x,y,color='red',alpha=0.3,linestyle='-  
,linewidth=5,marker='o',markeredgewidth=10)
```

- color:可以使用颜色的16进制，也可以使用线条颜色的英文，还可是使用之前的缩写那么，我们最终要挪动的其实就是这四个 “支柱”
- alpha: 0-1, 透明度
- linestyle:折线样式
- marker标记点:

+ 图表的样式参数设置

字符	颜色	英文全称
'b'	蓝色	blue
'g'	绿色	green
'r'	红色	red
'c'	青色	cyan
'm'	品红	magenta
'y'	黄色	yellow
'k'	黑色	black
'w'	白色	white

字符	描述
'-'	实线
'--'	虚线
'-.'	点划线
'.'	虚线

标记符号	描述
'.'	点标记
'o'	圆圈标记
'x'	'X'标记
'D'	钻石标记
'H'	六角标记
's'	正方形标记
'+'	加号标记

➤ 创建图形对象

- 在 Matplotlib 中，面向对象编程的核心思想是创建图形对象（figure object）。通过图形对象来调用其它的方法和属性，这样有助于我们更好地处理多个画布。在这个过程中，pyplot 负责生成图形对象，并通过该对象来添加一个或多个 axes 对象（即绘图区域）。
- Matplotlib 提供了 `matplotlib.figure` 图形类模块，它包含了创建图形对象的方法。通过调用 pyplot 模块中 `figure()` 函数来实例化 figure 对象。

➤ 创建图形对象

- figure方法如下：

```
plt.figure(  
    num=None, -----> 图像编号或名称，数字为编号，字符串为名称  
    figsize=None, -----> 指定figure的宽和高，单位为英寸；  
    dpi=None, -----> 定绘图对象的分辨率，即每英寸多少个像素，缺省值为72  
    facecolor=None, -----> 背景颜色  
    edgecolor=None, -----> 边框颜色  
    frameon=True, -----> 是否显示边框  
    **kwargs,  
)
```



1. 在 Python 中, `**kwargs` 用于接收任意的命名参数, 并把它们以字典形式传入函数 (键=参数名, 值=参数值)。
2. 在 matplotlib 的绘图函数 (如 `plt.plot`) 中, `**kwargs` 用来传入图形属性, 函数会把这些键值对转给底层的 `Line2D/Artist`, 用以控制颜色、线型、标记、透明度等外观。

常见 kwargs (对应你代码中的示例) :

`color / c`: 线或标记颜色 (例如 'red' 或 '#ff0000')

`alpha`: 透明度 (0-1)

`linestyle / ls`: 线型 ('-', '--' 等)

`linewidth / lw`: 线宽

`marker`: 标记样式 ('o', 's', 'H' 等)

`markersize / ms`: 标记大小

`markeredgecolor / mec`: 标记边颜色

`markeredgewidth / mew`: 标记边宽

`markerfacecolor / mfc`: 标记填充色

`label`: 图例标签

`zorder`: 绘制顺序

子图和柱状图内容

- 三种方式绘制子图
 - `add_axes`
 - `subplot`
 - `subplots`
- 柱状图

➤ 创建图形对象

- 在 Matplotlib 中，面向对象编程的核心思想是创建图形对象（figure object）。通过图形对象来调用其它的方法和属性，这样有助于我们更好地处理多个画布。在这个过程中，pyplot 负责生成图形对象，并通过该对象来添加一个或多个 axes 对象（即绘图区域）。
- Matplotlib 提供了 `matplotlib.figure` 图形类模块，它包含了创建图形对象的方法。通过调用 pyplot 模块中 `figure()` 函数来实例化 figure 对象。

➤ 创建图形对象

- figure方法如下：

```
plt.figure(  
    num=None, -----> 图像编号或名称，数字为编号，字符串为名称  
    figsize=None, -----> 指定figure的宽和高，单位为英寸；  
    dpi=None, -----> 定绘图对象的分辨率，即每英寸多少个像素，缺省值为72  
    facecolor=None, -----> 背景颜色  
    edgecolor=None, -----> 边框颜色  
    frameon=True, -----> 是否显示边框  
    **kwargs,  
)
```


➤ 绘制多子图

- figure是绘制对象(可理解为一个空白的画布), 一个figure对象可以包含多个Axes子图, 一个Axes是一个绘图区域, 不加设置时, Axes为1, 且每次绘图其实都是在figure上的Axes上绘图。
- 接下来将学习绘制子图的几种方式:
 - `add_axes()`: 添加区域
 - `subplot()`: 均等地划分画布,只是创建一个包含子图区域的画布,(返回区域对象)
 - `subplots()`: 既创建了一个包含子图区域的画布, 又创建了一个 figure 图形对象.(返回图形对象和区域对象)

➕ **add_axes()** : 添加区域

- Matplotlib 定义了一个 axes 类（轴域类），该类的对象被称为 axes 对象（即轴域对象），它指定了一个有数值范围限制的绘图区域。在一个给定的画布（figure）中可以包含多个 axes 对象，但是同一个 axes 对象只能在一个画布中使用。
- 语法:

```
add_axes(rect)
```

- 该方法用来生成一个 axes 轴域对象，对象的位置由参数 rect 决定
- rect 是位置参数，接受一个由 4 个元素组成的浮点数列表，形如 [left, bottom, width, height]，它表示添加到画布中的矩形区域的左下角坐标(x, y)，以及宽度和高度。

➤ 区域中基本方法的使用

- 区域图表名称: `set_title()`
- 区域中x轴和y轴名称:`set_xlabel()` `set_ylabel()`
- 刻度设置: `set_xticks()`
- 区域图表图例: `legend()`

➕ subplot() 函数，它可以均等地划分画布

- 参数格式如下：

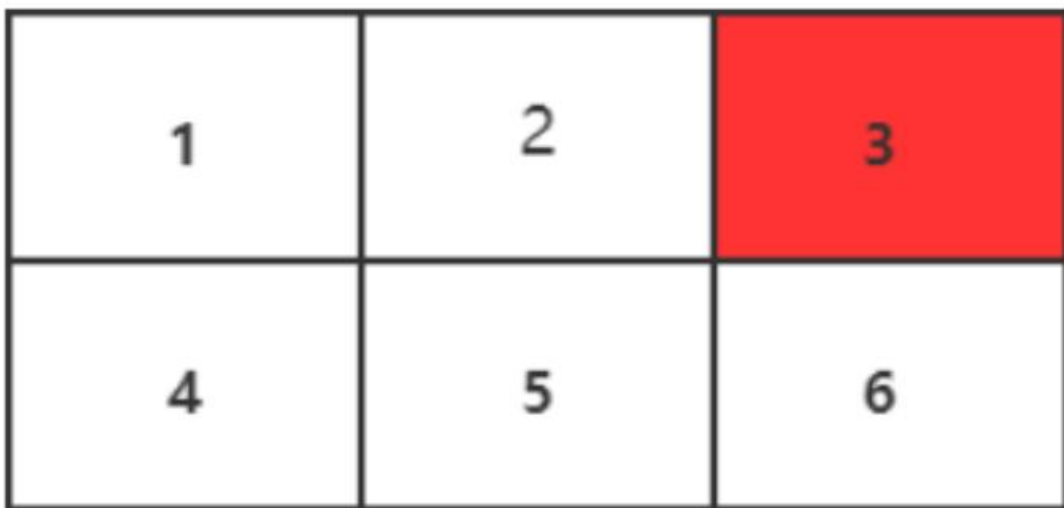
```
ax = plt.subplot(nrows, ncols, index,*args, **kwargs)
```

- nrows 行
- ncols 列
- index: 索引
- kwargs: title/xlabel/ylabel 等....

>也可以直接将几个值写到一起,如:subplot(233)

➕ subplot() 函数，它可以均等地划分画布

- `nrows` 与 `ncols` 表示要划分几行几列的子区域（`nrows*ncols`表示子图数量），`index` 的初始值为1，用来选定具体的某个子区域。
- 例如： `subplot(233)`表示在当前画布的右上角创建一个两行三列的绘图区域（如下图所示），同时，选择在第 3 个位置绘制子图。



➕ **subplots()函数详解**

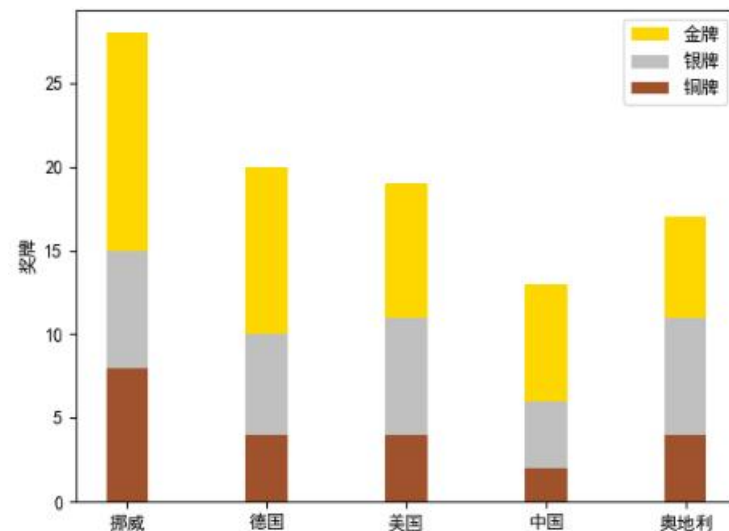
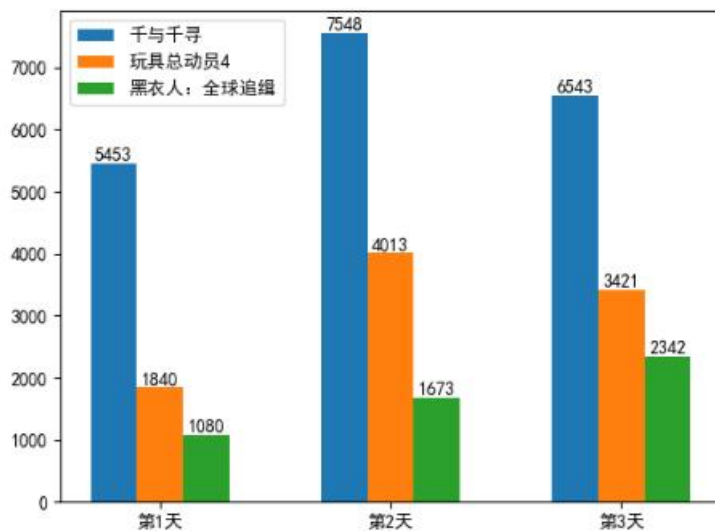
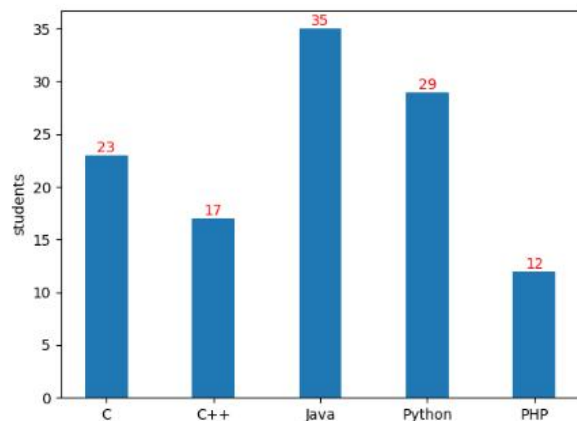
- matplotlib.pyplot模块提供了一个 `subplots()` 函数，它的使用方法和 `subplot()` 函数类似。其不同之处在于，`subplots()` 既创建了一个包含子图区域的画布，又创建了一个 `figure` 图形对象，而 `subplot()` 只是创建一个包含子图区域的画布。
- `subplots` 的函数格式如下：

```
fig , ax = plt.subplots(nrows, ncols)
```

- `nrows` 与 `ncols` 表示两个整数参数，它们指定子图所占的行数、列

+ 柱状图的绘制

- 柱状图是一种用矩形柱来表示数据分类的图表。
 - 柱状图可以垂直绘制，也可以水平绘制。
 - 它的高度与其所表示的数值成正比关系。
 - 柱状图显示了不同类别之间的比较关系，图表的水平轴 X 指定被比较的类别，垂直轴 Y 则表示具体的类别值。



➤ 柱状图的绘制

- 其语法格式如下

```
matplotlib.pyplot.bar(x, height, width=0.8, bottom=None, *, align='center', data=None, **kwargs)
```

- 参数说明

x	一个标量序列，代表柱状图的x坐标（当无需参与计算时，也可以是对应的标签序列）
height	一个标量或者是标量序列，代表柱状图的高度。
width	可选参数，标量或类数组，柱状图的默认宽度值为 0.8。
bottom	可选参数，标量或类数组，柱状图的y坐标默认为None。
align	有两个可选项 {"center","edge"}，默认为 'center'，默认x取值是每个柱状图所在的中点位置，或者也可以是柱状图左侧边缘位置

+ 普通柱状图

水平轴 X 指定被比较的类别，垂直轴 Y 则表示具体的类别值

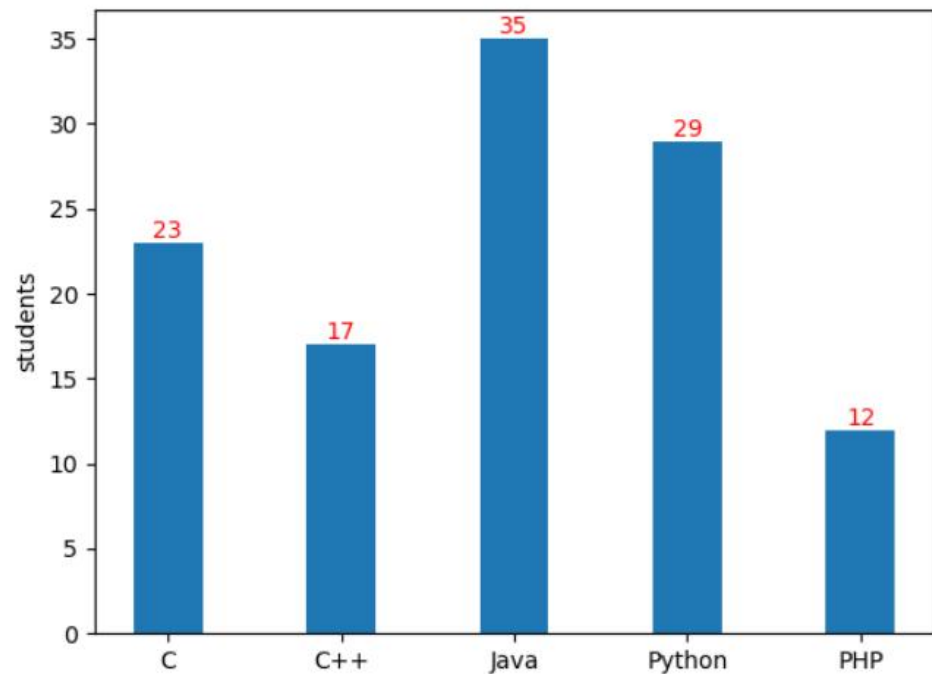
● 数据：

#一下提供2组数据，分别是学习语言和学生人数

```
langs = ['C', 'C++', 'Java', 'Python', 'PHP']
```

```
students = [23,17,35,29,12]
```

● 最终结果：



➕ 堆叠柱状图

- 分析：
 - 确定x轴和y轴的数据
 - 设置柱状图宽度，使用柱状图绘图方法绘制
 - 柱状图数值显示：使用文本添加方法进行文本内容添加

+ 普通柱状图

- 图形添加文本信息语法:

```
matplotlib.pyplot.text(x, y, s, **kwargs)
```

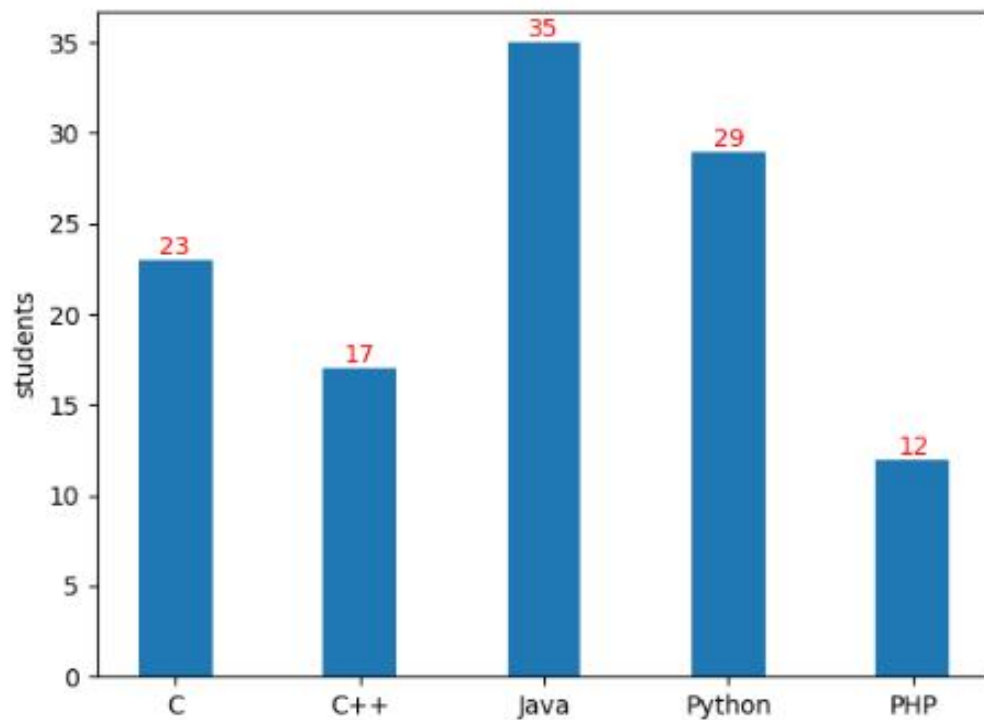
- 通过函数方式，向axes对象添加 text对象，确切的说是向axes的 (x , y)位置添加 s 文本。
返回一个text实例
- 参数说明:

x, y	确定文本内容的坐标位置
s	str 内容text
fontsize/fontweight/fontstyle	字体的相关设置：字体大小/字体粗细/字体类型
va	水平对齐方式，可选参数： 'center' , 'top' , 'bottom' , 'baseline'
ha	设置垂直对齐方式，可选参数： left,right,center
rotation	(旋转角度)可选参数

+ 普通柱状图

● 代码实现

```
from matplotlib import pyplot as plt
# 数据准备
langs = ['C', 'C++', 'Java', 'Python', 'PHP']
students = [23, 17, 35, 29, 12]
# 绘制图像
plt.bar(langs, students, width=0.4)
# 循环显示数据标签
for a, b in zip(langs, students):
    plt.text(a, b, b, color='red', ha='center', va='bottom')
# 设置y轴标签
plt.ylabel('students')
# 显示图像
plt.show()
```



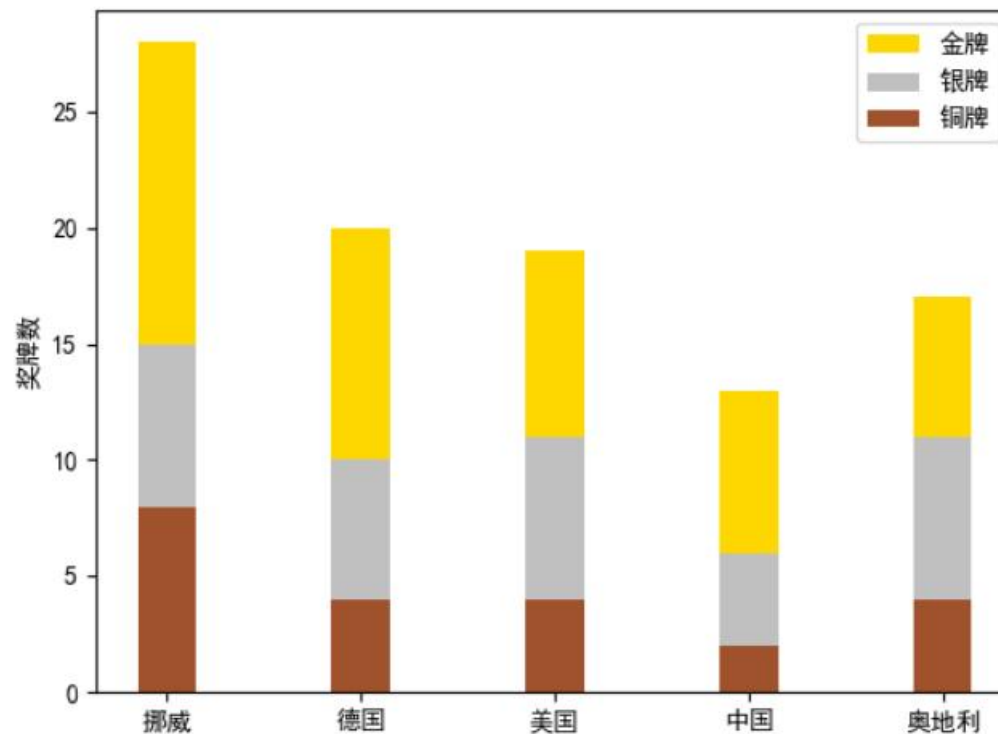
+ 堆叠柱状图

所谓堆叠柱状图就是将不同数组别的柱状图堆叠在一起，堆叠后的柱状图高度显示了两者的相加的结果值。

● 数据：

```
#截止16日，北京冬奥会前5名国家奖牌数  
# 前五名奖牌榜  
countries = ['挪威', '德国', '美国', '中国', '奥地利']  
# 金牌个数  
gold_medal = [13, 10, 8, 7, 6]  
# 银牌个数  
silver_medal = [7, 6, 7, 4, 7]  
# 铜牌个数  
bronze_medal = [8, 4, 4, 2, 4]
```

● 最终结果：



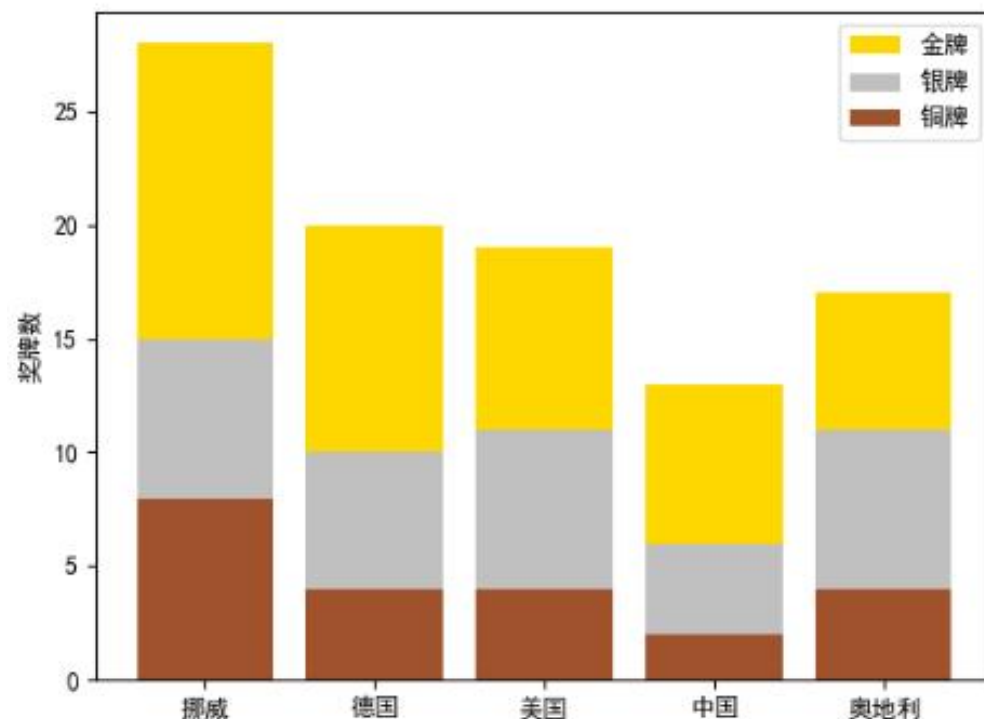
➕ 堆叠柱状图

- 分析：
 - 金牌榜的起始高度为：铜牌数据+银牌数据
 - 银牌榜的起始高度为：银牌高度
 - 铜牌榜的起始高度为：0
 - 起始位置的数据相加需要使用numpy的相关知识
 - 需要确定柱状图的颜色
 - 显示图例

+ 堆叠柱状图

● 代码实现

```
from matplotlib import pyplot as plt
import numpy as np
# 设置中文
plt.rcParams['font.sans-serif'] = ['SimHei']
# 前五名奖牌榜
countries = ['挪威', '德国', '美国', '中国', '奥地利']
# 金牌个数
gold_medal = [13, 10, 8, 7, 6]
# 银牌个数
silver_medal = [7, 6, 7, 4, 7]
# 铜牌个数
bronze_medal = [8, 4, 4, 2, 4]
# 绘制堆叠图
plt.bar(countries, gold_medal, color='gold', label='金牌',
        bottom=np.array(silver_medal) + np.array(bronze_medal))
plt.bar(countries, silver_medal, color='silver', label='银牌',
        bottom=bronze_medal)
plt.bar(countries, bronze_medal, color='#A0522D', label='铜牌')
# 设置坐标轴
plt.ylabel('奖牌数')
# 设置图例
plt.legend(loc='upper right')
# 显示
plt.show()
```



+ 同位置多柱状图

同一 x 轴位置绘制多个柱状图，主要通过调整柱状图的宽度和每个柱状图x轴的起始位置

● 数据：

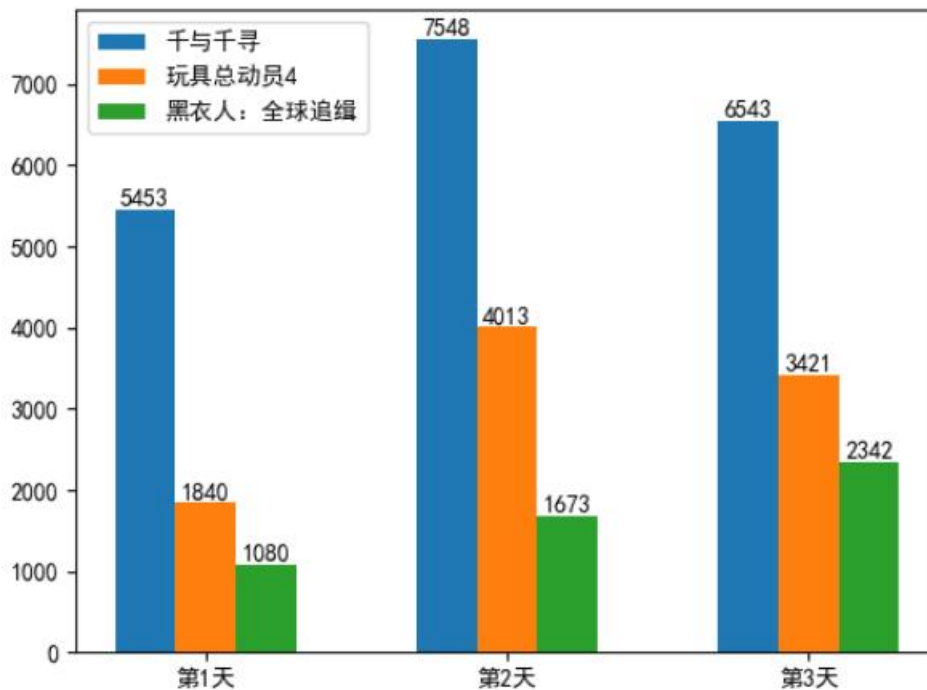
```
# 三天中3部电影的票房变化
movie = ['千与千寻', '玩具总动员4', '黑衣人：全球追缉']

real_day1 = [5453, 1840, 1080]

real_day2 = [7548, 4013, 1673]

real_day3 = [6543, 3421, 2342]
```

● 最终结果：



➕ 同位置多柱状图

- 分析：
 - 本实例需要对x轴进行计算，因此需要将x轴转数值
 - 确定同一x轴中，每个柱状图x轴的起始位置。
 - 需要设置图形的宽度
 - $\text{图形2的起始位置} = \text{图形2起始位置} + \text{图形的宽度}$
 - $\text{图形3的起始位置} = \text{图形3起始位置} + 2 \times \text{图形的宽度}$
 - 需要给每个柱状图循环显示文本内容
 - 显示图例

+ 同位置多柱状图

● 代码实现

```
import matplotlib.pyplot as plt
import numpy as np
# 设置中文
plt.rcParams['font.sans-serif']=['SimHei']
#三天中3部电影的票房变化
movie=['千与千寻','玩具总动员4','黑衣人：全球追缉']
real_num1=[5453,7548,6543]
real_num2=[1840,4013,3421]
real_num3=[1080,1673,2342]
x = np.arange(len(movie))
x_label = ['第{}天'.format(i+1) for i in x]
# 图形宽度
width = 0.2
# 图形2的起始位置
bar2_x = x + width
# 图形3的起始位置
bar3_x = x + 2*width
```

● 代码实现

```
# 绘图
plt.bar(x,real_num1,width=width,label=movie[0])
plt.bar(bar2_x,real_num2,width=width,label=movie[1])
plt.bar(bar3_x,real_num3,width=width,label=movie[2])
# 设置柱状图文本内容
for x,y in zip(x,real_num1):
    plt.text(x,y,y,ha='center',va='bottom')
for x,y in zip(bar2_x,real_num2):
    plt.text(x,y,y,ha='center',va='bottom')
for x,y in zip(bar3_x,real_num3):
    plt.text(x,y,y,ha='center',va='bottom')
# 更改x坐标标签
plt.xticks(bar2_x,x_label)
# 显示图例
plt.legend()
plt.show()
```